

SIH 2026 · PROBLEM STATEMENT SIH26140

QUANTIFY

Implementation Deep-Dive — Phase 1: Full Codebase Walkthrough

Prepared for: Kavya Chandegara — Project Lead & Frontend Engineering

Purpose: Complete functional map of every module, so every line of the codebase can be explained and defended with confidence.

Scope: Auth & sessions, database & RLS, all 10 Edge Functions, the quantum simulator engine (x3 implementations), AI tutor, gamification, admin panel, and frontend composition.

Companion to: Phase 2 (refactor for ownership) & Phase 3 (strengthen & test)

Table of Contents

- 0. How to Use This Document
- 1. What QUANTIFY Actually Is
- 2. Architecture at a Glance
- 3. Tech Stack
- 4. Project Structure — What Lives Where
- 5. Data Model
- 6. Auth & Session Flow
- 7. Feature-by-Feature Deep Dive
 - 7.1 Diagnostic Assessment — `submit-assessment`
 - 7.2 Adaptive Learning Path — `get-learning-path`
 - 7.3 Checkpoint Quizzes — `submit-quiz`
 - 7.4 Quanta AI Tutor — `tutor-chat`
 - 7.5 Quantum Circuit Simulator — the most important section to know
 - 7.6 Save / Load Circuits — `save-circuit` , `get-my-circuits`
 - 7.7 Achievements / Gamification
 - 7.8 Admin Panel
 - 7.9 Discussion Forum
- 8. Frontend Composition
- 9. What To Know Cold Before Judges Ask — Honest Findings
- 10. Quick-Reference Cheat Sheet

0. How to Use This Document

This is not a README rewrite — it is a working map of **every functional layer** of the QUANTIFY codebase, written so you can answer any “how does this actually work?” question from a judge without hesitation. Each section explains **what the code does, why it’s structured that way, and what to say if you’re asked about it**. Section 9 is the most important one to read before demo day — it lists the parts of the codebase that are genuinely weak or inconsistent, found by tracing the code directly rather than assuming the README is accurate.

Everything below was verified against the actual source files in `SIH_2026_79-main/`, not inferred from the README.

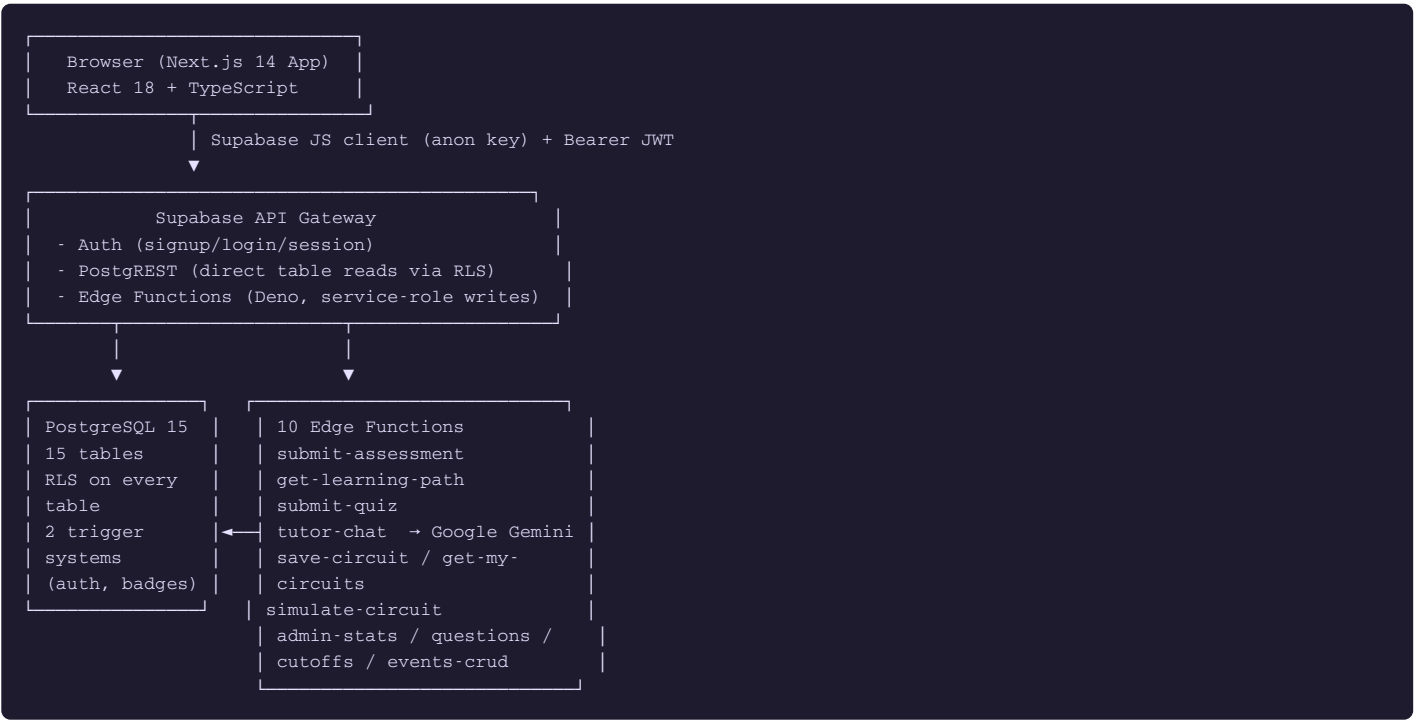
1. What QUANTIFY Actually Is

QUANTIFY is a **Next.js 14 (App Router) + Supabase** web application that teaches quantum computing through five connected systems:

- 1. A **diagnostic assessment** that grades a learner across 5 categories and assigns a proficiency tier.
- 2. An **adaptive learning path** that sequences 13 curriculum topics based on that tier and any weak categories.
- 3. **Per-topic checkpoint quizzes** that unlock progress when passed.
- 4. A **drag-and-drop quantum circuit simulator** with a real statevector math engine (not a mock).
- 5. A **Gemini-powered AI tutor** (“Quanta”) with conversation memory, plus a **discussion forum**, **achievements/gamification layer**, and an **admin back-office**.

The backend is entirely **Supabase**: Postgres 15 for storage, Supabase Auth for login, Row Level Security (RLS) for per-user data isolation, and **Deno-runtime Edge Functions** for all server-side logic (grading, AI calls, circuit simulation). There is a parallel Python script (`qiskit_simulator.py`) intended for heavier simulation, described honestly in Section 7.5.

2. Architecture at a Glance



Key architectural decision to understand cold: almost nothing is graded or computed on the client. Assessment scoring, quiz scoring, learning-path sequencing, and circuit simulation (the edge-function version) all happen server-side in Edge Functions using the Supabase **service role key**, which bypasses RLS entirely. This is a deliberate “zero-trust” pattern — the client only ever submits raw answers, never a score — but it also means the Edge Functions themselves are the actual security boundary for writes, not just RLS (see Section 9.3).

3. Tech Stack

Layer	Technology	Notes
Frontend framework	Next.js 14.2 (App Router)	File-based routing under <code>app/</code>
Language	TypeScript 5.4	Strict mode; <code>next.config.mjs</code> does not suppress build errors
UI	React 18, Tailwind CSS 3.4	Glassmorphism visual style, Lucide icons
Charts	Recharts 3	Used in dashboard & admin telemetry
Auth & DB	Supabase (<code>@supabase/supabase-js</code> , <code>@supabase/ssr</code>)	Postgres 15 + Auth + RLS
Serverless	Supabase Edge Functions (Deno)	10 functions, one folder per function
AI	Google Gemini API (REST, called directly via <code>fetch</code>)	Model fallback chain, not an SDK
Quantum math (client)	Hand-written complex-number statevector engine	<code>lib/quantum-simulator.ts</code>
Quantum math (edge)	Same algorithm, duplicated in Deno/TS	<code>simulate-circuit</code> function
Quantum math (offline script)	Python + NumPy (Qiskit imported but unused)	<code>backend/qiskit_simulator.py</code>
Image export	html2canvas	Achievement/result share cards

4. Project Structure — What Lives Where

```

app/
├─ dashboard/           → 17 routed pages (App Router), one folder per route
├─ assessment/          progress overview, charts
├─ path/                diagnostic quiz flow
├─ topic/[id]/          adaptive roadmap
├─ quiz/[id]/           theory + Bloch sphere + video per topic
├─ simulator/           per-topic checkpoint quiz
├─ tutor/               circuit builder (725 lines — largest page)
├─ achievements/        Quanta AI chat UI
├─ discussions/         badge gallery
├─ admin/               forum (list + [id] thread view)
├─ events/, resources/, books/, challenges/, profile/, onboarding/, login/
└─ layout.tsx, page.tsx (landing page), template.tsx

backend/
├─ schema.sql           → "source of truth" copy of DB + Edge Function code
├─ rls_policies.sql      15 tables, indexes, comments
├─ rpc_functions.sql     RLS for every table + is_admin() helper
├─ achievements_trigger.sql  Postgres stored procedures (e.g. daily-challenge streaks)
├─ auth_trigger.sql      badge unlock engine
├─ qiskit_simulator.py   auto-provisions a profile row on signup
├─ functions/           standalone Python simulation script
└─                      10 Edge Function source folders (mirrored in supabase/functions/)

supabase/               → Supabase CLI deployment copy (same 10 functions + 3 extra migrations)
lib/
├─ auth-context.tsx     global user state (React Context)
├─ quantum-simulator.ts client-side statevector engine
├─ qiskit-parser.ts     parses/validates circuit JSON
├─ api/                 one typed wrapper file per Edge Function (assessment.ts, quiz.ts, tutor.ts, circuits.ts, ...)
└─ mock/                static fallback data per feature (used when Supabase is unreachable)

components/
├─ layout/AppShell.tsx  sidebar nav + header, wraps every authenticated page
├─ simulator/           gate tooltip, intro overlay
├─ discussions/         12 components — full forum UI kit
└─ ui/                  EmptyState, LoadingState, Skeleton primitives

types/quantify.ts       → central domain types (GateType, PlacedGate, SimulationResult, ...)
types/discussion.ts     → forum domain types
middleware.ts           → route-level auth gate

```

Why this matters for judges: the `lib/api/*.ts` files are a clean seam — every page calls a typed function (`submitAssessment()` , `getLearningPath()` , `simulateCircuit()` , ...) instead of calling `fetch` directly. If asked “how is your frontend decoupled from your backend,” point here.

5. Data Model

15 Postgres tables, **RLS enabled on every single one** (verified in `rls_policies.sql` , not just claimed in the README):

Table	Purpose	Access rule
<code>profiles</code>	1:1 with <code>auth.users</code> ; level, streak, quiz average, weak/strong topics, bookmarks, badges	Own row only, admin sees all
<code>assessment_questions</code>	Diagnostic question bank (JSONB <code>options</code> array w/ <code>isCorrect</code>)	Public read, admin write
<code>assessment_attempts</code>	Audit trail of every diagnostic submission	Own rows only
<code>topics</code>	13 curriculum modules (category, level, sequenceOrder, theory, video, formulas)	Public read, admin write
<code>learning_path_items</code>	(present in schema; the live path is actually <i>computed on the fly</i> by the edge function — see 7.2)	Own rows only
<code>quiz_questions</code> / <code>quiz_attempts</code>	Per-topic checkpoint questions and submission history	Questions public, attempts own-only
<code>saved_circuits</code>	Serialized circuit (gates + result JSON) per user	Own rows only
<code>tutor_conversations</code> / <code>tutor_messages</code>	Chat thread + message history for Quanta AI	Own rows only
<code>resources</code> , <code>books</code>	Curated learning library	Public read, admin write
<code>achievements</code> , <code>user_achievements</code>	Badge catalog + per-user unlock log	Catalog public, unlocks own-only
<code>platform_settings</code>	Admin-configurable tier cutoffs (<code>beginnerMax</code> , <code>intermediateMax</code>)	Admin write, used by grading logic

The `is_admin()` pattern: every admin-bypass rule uses a `SECURITY DEFINER` Postgres function that checks `profiles.role = 'admin'`. This avoids infinite RLS recursion (a table’s policy can’t safely query itself). This is a legitimately good pattern — be ready to explain *why* `SECURITY DEFINER` is needed here if asked.

Two automated trigger systems, both fire on Postgres events (not application code):

- `handle_new_user()` (`auth_trigger.sql`) — fires on `auth.users` insert, auto-creates the matching `profiles` row with defaults.
- `check_and_unlock_achievements()` (`achievements_trigger.sql`) — fires after assessment/quiz/circuit/streak events, evaluates 8 badge conditions, has an anti-recursion guard so unlocking a badge doesn’t re-trigger itself.

6. Auth & Session Flow

Three pieces work together:

`middleware.ts` — runs on every request to a protected route (`/dashboard` , `/simulator` , `/admin` , etc., listed explicitly in `config.matcher`). It has a *fast path*: if a `quantify_session` or any `*supabase*` / `*auth-token*` cookie is present, it skips the network round-trip and lets the request through immediately. Only if no session-like cookie exists does it actually call `supabase.auth.getUser()` server-side and redirect to `/login` on failure.

`lib/auth-context.tsx` — a React Context (`AuthProvider`) wrapping the whole app. On mount it does three things in order: (1) hydrate from a `quantify_user_profile` `localStorage` cache for instant UI, (2) call `supabase.auth.getSession()` and, if logged in, fetch the real row from `profiles` and overwrite state with it, (3) subscribe to `onAuthStateChanged` so sign-in/sign-out events keep state in sync. `DEFAULT_USER` is a fully-populated demo profile (“Alex Vance”) used before any real session resolves — this is what a logged-out or first-render user sees, not a bug.

`auth_trigger.sql` — the moment Supabase Auth creates a row in `auth.users` (i.e., right after signup), a Postgres trigger inserts the matching `profiles` row server-side. The frontend never has to manually create a profile after signup.

Talking point: the `localStorage` cache in `auth-context.tsx` is a UX optimization (instant paint, no loading spinner flash), not the source of truth — it’s always overwritten by the live Supabase session moments later.

7. Feature-by-Feature Deep Dive

7.1 Diagnostic Assessment — `submit-assessment`

Client sends `{ answers: { questionId: optionId } }`. The function:

1. Authenticates the caller (accepts `userId` in body **or** derives it from the `Authorization: Bearer` JWT).
2. Re-fetches **all** questions from `assessment_questions` server-side — the correct answers are never trusted from the client.
3. Grades each question by comparing the submitted option ID against the option flagged `isCorrect: true` inside the question's JSONB `options` array.
4. Buckets results into 5 fixed categories (`basics`, `qubits`, `gates`, `circuits`, `algorithms`) via keyword matching on the question's stored `category` string (`normalizeCategory()`).
5. Reads tier cutoffs from `platform_settings` (defaults: $\leq 3 \rightarrow$ beginner, $\leq 7 \rightarrow$ intermediate, else advanced) — **these cutoffs are admin-configurable at runtime**, which is why the admin panel has a “cutoffs” screen.
6. Inserts an audit row into `assessment_attempts` and updates `profiles.level` / `weakTopics` / `strongTopics` / `assessmentCompleted`.

This is the most defensible piece of backend logic in the app — genuinely zero-trust grading. Know this function cold.

7.2 Adaptive Learning Path — `get-learning-path`

Important nuance: despite `learning_path_items` existing as a table, **the path is not read from that table** — it's recomputed from scratch on every call:

1. Fetch the user's `level`, `completedTopics`, `weakTopics` from `profiles`.
2. Fetch all 13 `topics`, ordered by `sequenceOrder`.
3. For each topic: mark `completed` if in `completedTopics`; the **first non-completed topic becomes `in_progress`**; everything after that is `available` if the user's tier (`levelRank`) is $\geq$ the topic's tier (or it's within 2 positions of their progress), otherwise `locked`.
4. Any topic whose category/name fuzzy-matches a weak diagnostic category gets `sequenceOrder - 100`, which bubbles it to the top after sorting — that's the “priority revision” mechanic.

Talking point: this is a *stateless recomputation* pattern, not a stored roadmap — meaning if the curriculum (`topics` table) or the user's weak areas change, the path updates automatically next load with no migration needed.

7.3 Checkpoint Quizzes — `submit-quiz`

Same zero-trust pattern as assessment: server re-fetches `quiz_questions` for the given `topicId` (falling back to a `topicId = 'default'` question set if none exist yet for that topic), grades server-side, and supports matching the user's answer either by option `id` or by raw index for flexibility. Pass mark is a **hardcoded 70%**. On pass, it appends the topic to `completedTopics` and recalculates `overallProgress` as `completedTopics.length / totalTopicsInDB` — meaning progress-bar math depends on however many topics currently exist in the database, not a fixed constant.

7.4 Quanta AI Tutor — `tutor-chat`

This is the most elaborate function (335 lines). Flow per message:

1. Resolve `userId`, ensure a `tutor_conversations` row exists (upsert).
2. Persist the incoming user message to `tutor_messages` immediately.
3. Pull the **last 10 messages** for that conversation to build short-term memory, collapsing consecutive same-role messages into one turn (Gemini's `contents` array requires alternating roles).
4. Choose a system prompt based on `mode`: normal tutoring, `'analogy'` (short real-world analogy for the student's *current circuit*), or `'mistake-doctor'` (explains why a specific quiz answer was wrong). If a circuit is active, its qubit count / gates / generated code are injected into the prompt as grounding context.
5. Calls the Gemini REST endpoint directly with `fetch` (no SDK), trying a **fallback chain of model names** in order and moving to the next on a 429 or error.
6. If every model fails, it does **not** error out — it returns a hardcoded but topic-aware fallback message (different text for rate-limit vs. generic failure) so the UI never shows a broken state.
7. Extracts a code snippet (regex on triple-backtick blocks) and a “concept card” (first bolded phrase + first sentence) from the AI's raw text for the UI to render as structured widgets, then saves the assistant reply to `tutor_messages`.

Talking point: the fallback-message system means the tutor degrades gracefully under Gemini rate limits instead of showing an error screen — worth demoing on purpose if you can trigger it.

7.5 Quantum Circuit Simulator — the most important section to know

There are **three separate implementations** of the same statevector math. Know exactly which one runs where:

(a) `lib/quantum-simulator.ts` — runs in the browser. Pure math: represents each amplitude as `{r, i}` (real/imaginary), applies 2×2 unitary matrices for `X, Y, Z, H, S, T`, and bit-manipulation for `CNOT / SWAP` (flip the target index only where the control bit is 1). Qubit 0 is the **least-significant bit** (rightmost) in this file. Also computes a heuristic `isEntangled` flag (true only if a CNOT was used *and* multiple basis states each carry >30% probability). This is what powers the **live preview** as you drag gates onto the canvas — no network round-trip.

(b) `backend/functions/simulate-circuit/index.ts` (Deno Edge Function) — algorithmically the same statevector evolution, but qubit 0 is treated as the **most-significant bit** here (`qubitCount - 1 - t`), the opposite convention from (a). It also generates an OpenQASM 2.0 string and an executable Qiskit Python script as **text templates** (not run — just formatted strings), and labels its response `"backend": "Qiskit Aer Simulator"`.

(c) `backend/qiskit_simulator.py` — a standalone script. It **imports** `qiskit` and sets a `HAS_QISKIT` flag, but that flag is never actually used to branch into real Qiskit execution — `simulate_fallback()` (manual NumPy statevector math) runs unconditionally. So real IBM Qiskit Aer is never actually invoked anywhere in the running app; it's a well-structured placeholder for a future real integration. This script also isn't wired into any Edge Function — it's currently invoked manually (`python backend/qiskit_simulator.py '<json>'`), not by the deployed web app.

Talking point if asked “do you actually run Qiskit Aer?” Be straightforward: the UI and one edge function *label* the simulation backend “Qiskit Aer” for product-facing clarity, but the deployed simulation math is a from-scratch statevector engine (mirroring Qiskit Aer's output format — QASM + Python code generation) rather than a live Qiskit process. This is a completely legitimate engineering choice — writing your own lightweight simulator for a low-latency browser demo is *reasonable*, and you should own that decision rather than get caught off guard by the labeling.

`components/simulator/` (`GateTooltip.tsx`, `SimulatorIntro.tsx`) plus `app/simulator/page.tsx` (725 lines — the single largest page in the app) implement the drag-and-drop canvas, gate palette, step columns, and results panel (basis-state bar chart, Bloch-sphere-ish output, QASM/Python export tabs).

7.6 Save / Load Circuits — `save-circuit`, `get-my-circuits`

`save-circuit` upserts into `saved_circuits` keyed by a client-supplied or generated `id`; `get-my-circuits` fetches the caller's saved circuits plus any rows flagged `isPreset = true` (the built-in Bell State / Grover / Deutsch-Jozsa examples mentioned in the README). See Section 9.3 for an ownership-check gap worth knowing about here.

7.7 Achievements / Gamification

`achievements_trigger.sql` seeds 8 badges (`badge-1 ... badge-8`) and defines `check_and_unlock_achievements(user_id)`, called after the relevant table inserts/updates. Conditions include: first assessment taken, 3 topics mastered, a 2-qubit circuit built, a 7-day streak, 3 consecutive 100% quizzes, building Grover/Deutsch-Jozsa, 10+ tutor interactions, and 100% path completion. Unlocking writes to both `user_achievements` (log) and `profiles.unlockedBadges` (denormalized array for fast reads).

7.8 Admin Panel

Four Edge Functions back `/app/admin`:

- `admin-stats` — aggregate telemetry: active learner counts, average diagnostic score, per-category score distribution.
- `admin-questions-crud` — full CRUD over `assessment_questions` (largest admin function, 363 lines).
- `admin-cutoffs` — reads/writes the `platform_settings.beginnerMax / intermediateMax` values that `submit-assessment` consumes live.
- `admin-events-crud` — CRUD for the “Quantum Hub” events feed shown on `/events`.

Admin-only access is enforced two ways simultaneously: `middleware.ts` gates the `/admin/*` route generally (must be logged in), and `is_admin()`-based RLS policies gate the actual data (only rows readable/writable if `profiles.role = 'admin'`).

7.9 Discussion Forum

A full Q&A/discussion board (`types/discussion.ts`, 12 components under `components/discussions/`): categories, tags, threaded one-level replies, upvotes, and a report/moderation flow (`ReportModal.tsx`, `ReportStatus`). Modeled closely on a Stack-Overflow-style schema: `DiscussionSummary` (list view, 200-char preview) vs. full `Discussion` (detail view) — a sensible split to keep list-page payloads small.

8. Frontend Composition

- `AppShell.tsx` wraps every authenticated page: a collapsible sidebar with 13 nav items (Dashboard, Learning Path, Learn, Resources, Books, Simulator, Quanta AI, Quizzes, Achievements, Community, Profile, Admin Portal [conditional on `isAdmin`], Quantum Hub), a notification bell, and mobile hamburger state.
 - `lib/mock/*` — one fallback dataset per feature area, used by `lib/api/*` wrappers when a Supabase call fails, so the UI has something to render instead of a blank screen during a demo even if the network drops.
 - `context/theme-context.tsx` — light/dark theme provider, independent of `auth-context`.
 - **Custom hooks** (`hooks/`): `useCountUp` (animated stat numbers), `useInView` (scroll-triggered animations), `useScrollPosition`.
-

9. What To Know Cold Before Judges Ask — Honest Findings

These were found by reading the code directly, not by guessing. None of them are disqualifying — they’re normal things a real team’s codebase has — but you should be able to explain each one in one sentence rather than be surprised by it.

9.1 — Qubit-indexing convention differs between the two live simulators. `lib/quantum-simulator.ts` (client) treats qubit 0 as the least-significant bit; `simulate-circuit` (Edge Function) treats qubit 0 as the most-significant bit. Both are internally consistent and produce correct physics *within themselves*, but a circuit simulated client-side and the same circuit simulated server-side could label basis states with a different bit order for circuits that aren’t symmetric. Worth deciding which one is canonical and aligning the other — good, real Phase-3 fix.

9.2 — “Qiskit Aer” is a product label, not a live dependency. Covered in 7.5. The response field `"backend": "Qiskit Aer Simulator"` is a string constant, not a report of what actually ran. Real `qiskit` is imported in the Python script but never actually called.

9.3 — `save-circuit` doesn’t verify the caller owns the circuit id they’re upserting. Because the Edge Function authenticates to Postgres with the **service role key** (which bypasses RLS), the usual “users can only touch their own rows” protection doesn’t apply inside this function — it relies on the function’s own logic instead. Currently the function accepts any client-supplied `id` and upserts it under the caller’s `userId` without first checking that an existing row with that `id` already belongs to them. In practice this mainly matters if IDs are guessable (they’re UUIDs, so low risk), but it’s a real gap worth a one-line fix: `SELECT user_id FROM saved_circuits WHERE id = ?` before the upsert.

9.4 — `middleware.ts` has a hardcoded fallback Supabase URL and anon key. If the `NEXT_PUBLIC_SUPABASE_URL` / `NEXT_PUBLIC_SUPABASE_ANON_KEY` env vars are missing, the middleware falls back to literal values baked into the source. The anon key is meant to be public (that’s its purpose in Supabase’s model — it’s useless without RLS, which is enabled everywhere here), so this isn’t a security hole, but it does mean the app would silently point at a specific hosted project even with no `.env.local` configured. Worth knowing so you don’t get flustered if a judge notices a URL/key in the source.

9.5 — The repo already contains a self-audit (`ERROR_AUDIT_REPORT.md`). It documents 9 `any`-typed casts (listed with exact file/line) and 8 minor runtime edge cases, all verified as handled safely — e.g., the “Explain Like I’m New” button doesn’t disable itself for an empty circuit (wastes an API call, doesn’t crash), and same-date-prefix Supabase migrations are confirmed to still apply in safe alphabetical order. This report is useful ammunition — it shows the team already stress-tested edge cases rather than being unaware of them. Read through it once so you can cite specific lines if asked “what are your known issues?”

9.6 — `learning_path_items` table exists but isn’t the live data source (Section 7.2). Not a bug — `get-learning-path` recomputes on every call instead. But if a judge reads the schema and asks “why is this table basically empty,” now you know why.

10. Quick-Reference Cheat Sheet

If asked...	Point to
“Walk me through what happens when a student signs up”	<code>auth_trigger.sql</code> → <code>auth-context.tsx</code> → <code>middleware.ts</code>
“How is grading secured against cheating?”	<code>submit-assessment</code> / <code>submit-quiz</code> — server re-fetches correct answers, never trusts client
“Explain your database security model”	<code>rls_policies.sql</code> + <code>is_admin()</code> SECURITY DEFINER pattern
“How does the AI tutor stay on-topic?”	System prompt construction + circuit-context injection in <code>tutor-chat</code>
“Show me the actual quantum math”	<code>lib/quantum-simulator.ts</code> — complex number gate application
“What happens if Gemini is down?”	Fallback message logic in <code>tutor-chat</code> , Section 7.4 step 6
“What’s your hardest known limitation?”	Section 9.1 (qubit indexing) or 9.2 (Qiskit labeling) — answer honestly, then say how you’d fix it
“Who built what?”	README’s authorship table — you are listed as Project Lead & Frontend: system architecture, Simulator UI, AppShell, git/deployment coordination

End of Phase 1 document. Next: Phase 2 will go module-by-module through the code you’re going to rename/restructure/comment for real ownership, starting with `lib/quantum-simulator.ts` and the simulator UI — your named contribution area.